# Introduction to Tree

In computer science and data structures, trees are hierarchical structures consisting of nodes linked by edges. They are commonly used for hierarchical organization and data representation. This section delves into trees, their various types, and basic concepts.

## 1. Introduction to Trees

### Defining a Tree

A tree is made up of nodes that are arranged in a hierarchical structure. At the top of the hierarchy is the root node, which acts as the starting point. All other nodes are connected through edges. The nodes are grouped into levels, and the maximum level of any node in the tree is referred to as the depth.

### Fundamental Concepts

- **Root Node**: The node at the top of the tree. It is the node from where the whole tree originates. For any tree traversal operation, this node serves as the starting point.
- **Nodes**: All the elements in the tree, including the root, are nodes, and each node has a unique value and may have child nodes connected to it.
- **Parent Node**: A node in a tree that has one or more child nodes connected to it.
- **Child Node**: Nodes that are directly connected to a parent node.
- **Sibling Nodes**: Nodes that share the same parent node.
- **Ancestor Nodes**: All the nodes in the path from a specific node to the root node.
- **Descendant Nodes**: All the nodes reachable from a specific node down to the leaves.
- **Leaf Node**: Nodes in the tree that do not have any children.
- **Subtree**: A smaller tree within the main tree, consisting of a node and its descendants.

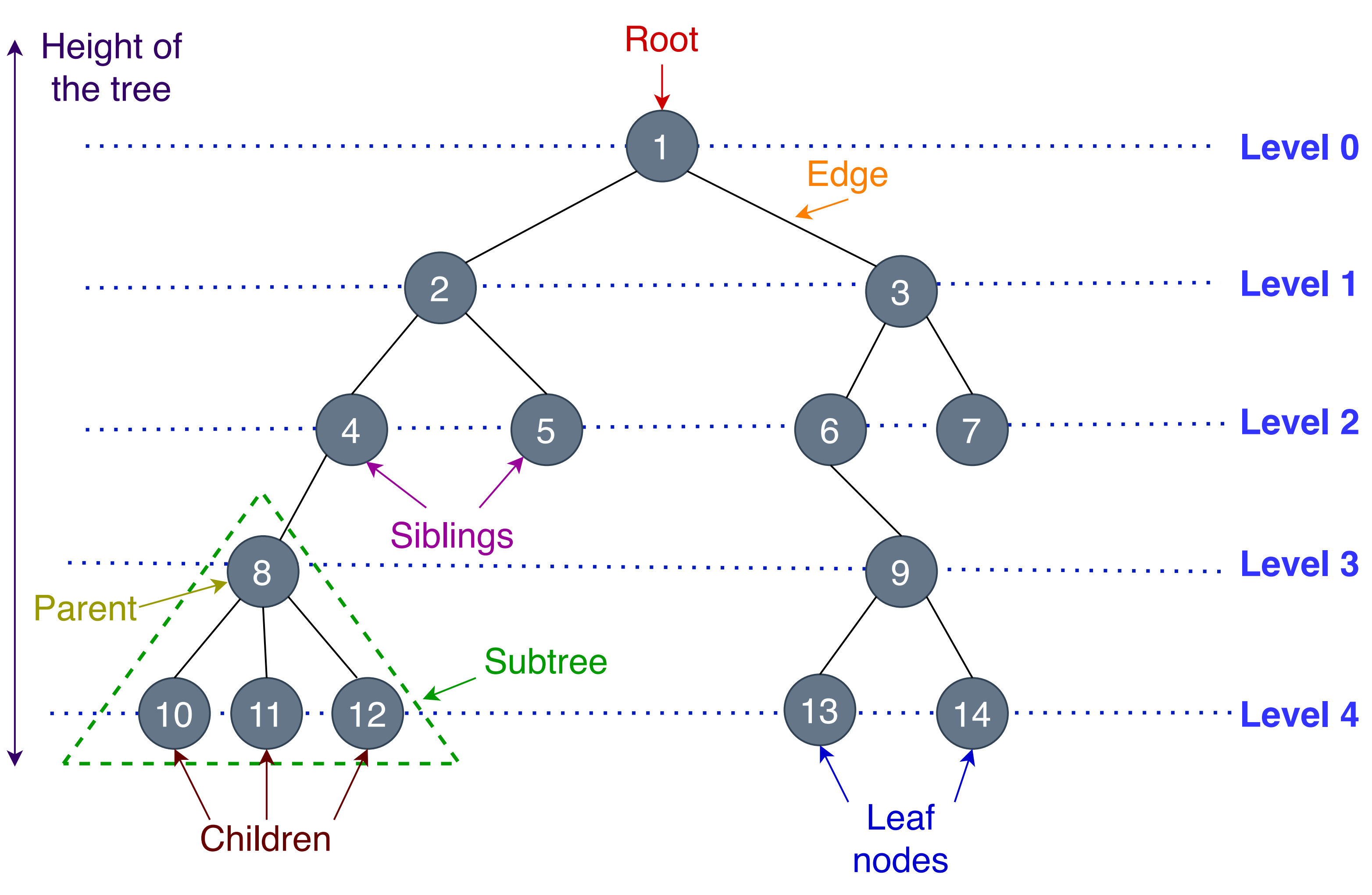
### Tree Height and Depth:

The height of a tree is the number of edges on the longest path from the root node to the leaf node. It represents the depth of the tree from the root. In contrast, the depth of a specific node in the tree is the number of edges from the root node to that particular node.

### Levels in Trees:

Levels in a tree are defined based on the distance from the root node. The root node is at level 0; its children are at level 1, and so on. The level of a node indicates its generation within the tree.

The following figure illustrates the different components of a tree.



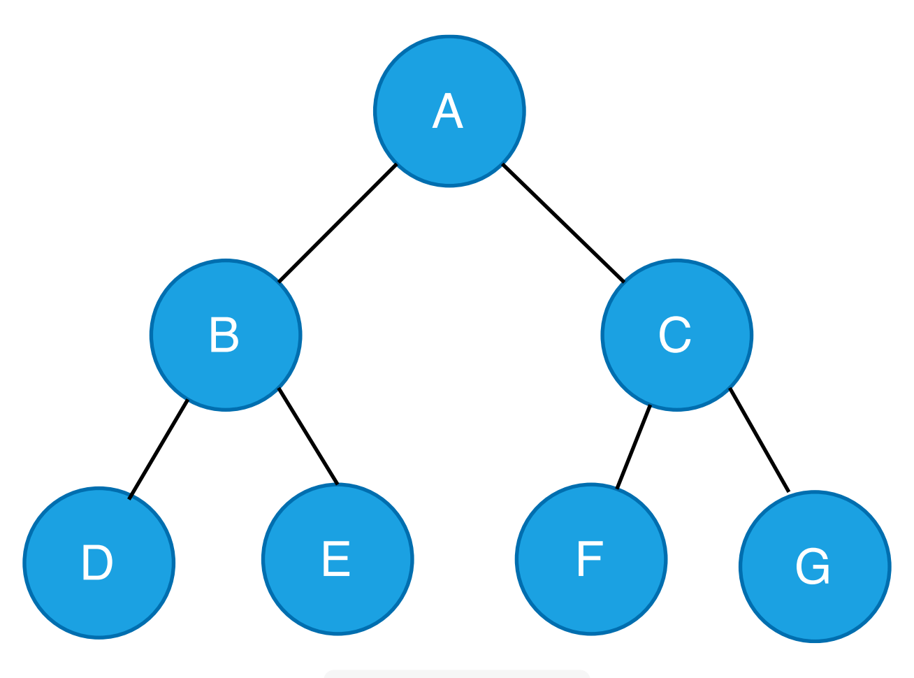
Binary Tree Basics

## Types of Tree

### Binary Trees:

Binary trees are a type of tree where each node can have at most two children, commonly referred to as the left child and the right child. The structure of a binary tree makes it a fundamental and widely used data structure in computer science.

### Example:



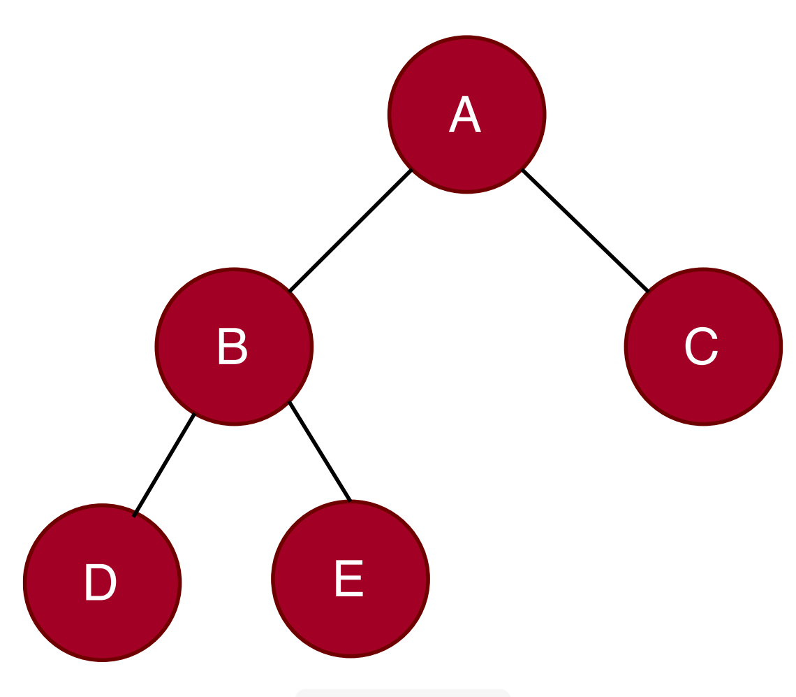
Binary Tree

In this example, "A" is the root node with two children, "B" ( the left child) and "C" ( the right child). "B" also has two children, "D" (left child) and "E" (right child). "C" also has two children, "F" (left child) and "G" (right child). Nodes "D," "E," "F," and "G" are leaf nodes as they do not have any children.

### Full Tree:

In a full tree, every node has either zero children (leaf node) or two children. There are no nodes with only one child. Full trees are also known as proper binary trees.

### Example:



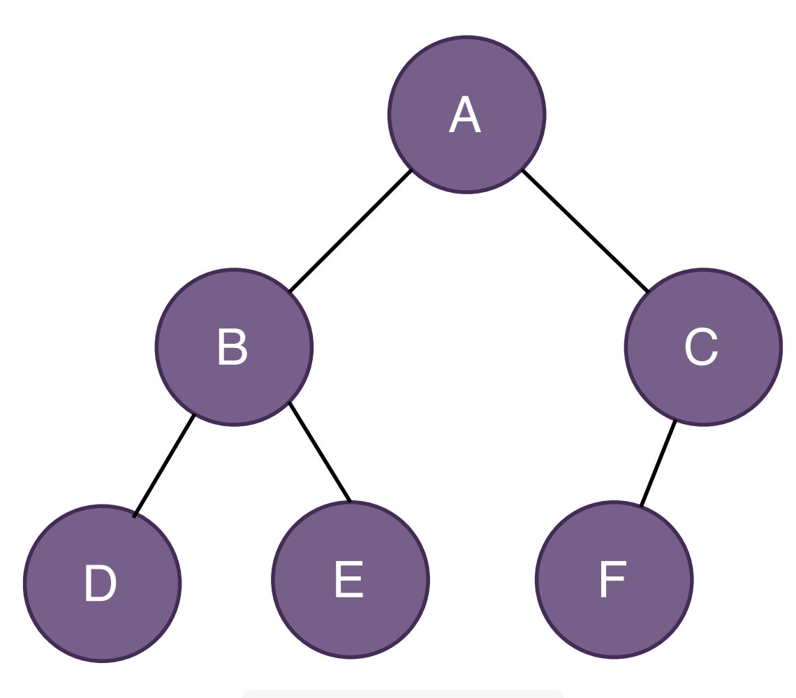
Full Tree

In this example, every node in the binary tree "A," "B," "C," "D," and "E" has either zero children (leaf nodes) or two children. It is a full tree.

### Complete Tree:

A complete tree is a binary tree in which all levels are filled, except possibly the last level. The last level must strictly be filled from left to right. Data structures like Heap uses complete binary trees for efficient operations.

### Example:



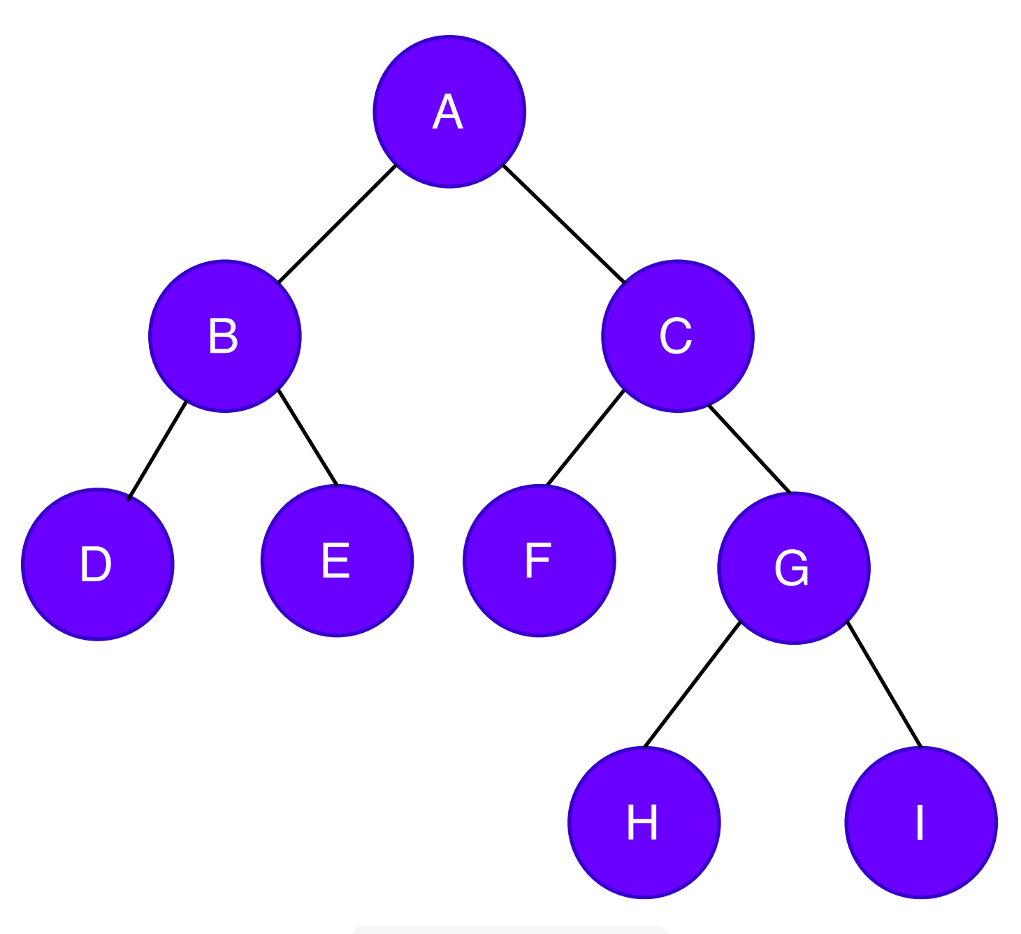
Complete Tree

In this example, the binary tree is complete because all levels, except the last level, are filled. The last level is filled from left to right with nodes "D," "E," and "F."

### Balanced Tree:

Balanced trees are binary trees where the difference in height between the left and right subtrees of any node in the tree is not more than 1. This ensures the tree remains reasonably balanced, preventing skewed structures and maintaining optimal search and insertion times.

### Example:



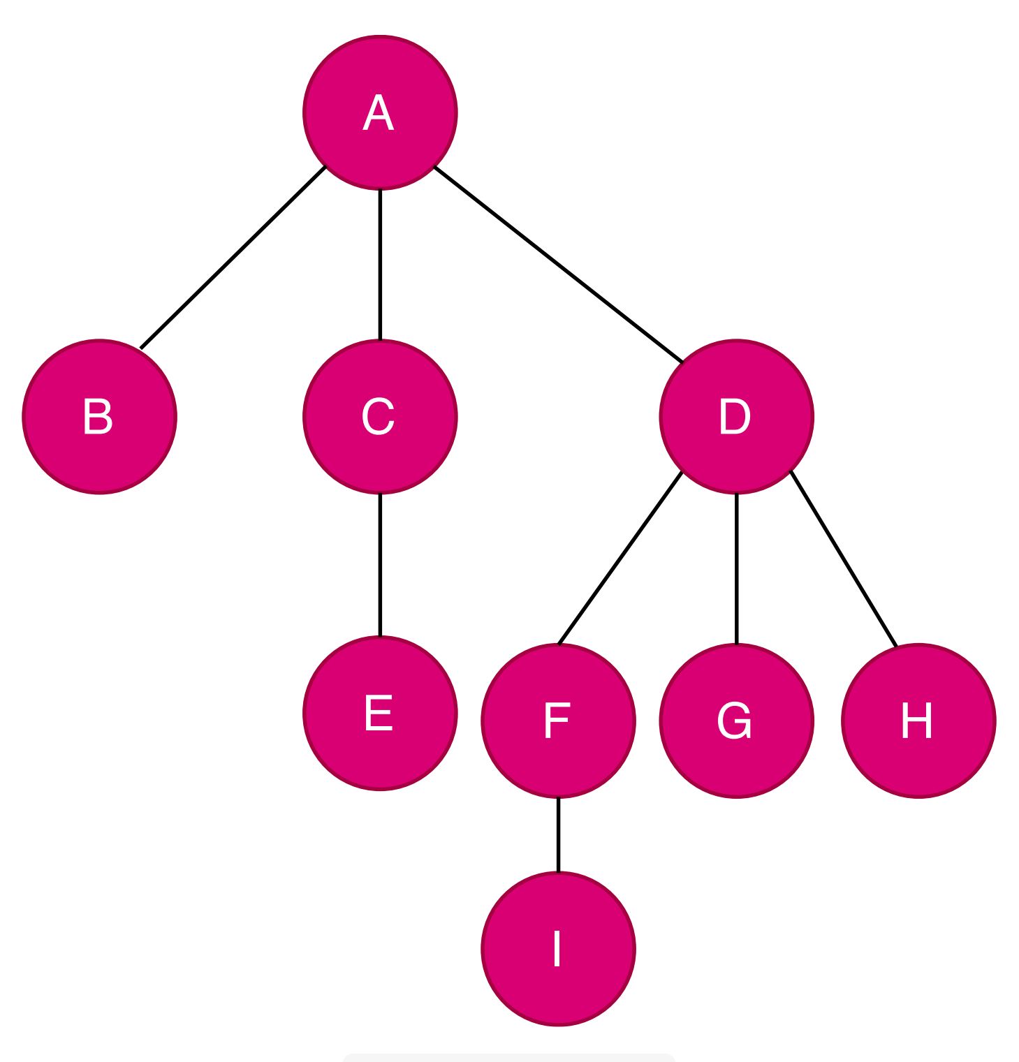
Balanced Tree

In this example, the binary tree is balanced. The height difference between the left and right subtrees of any node is not more than 1.

### Multi-way Tree

Unlike binary trees, multi-way trees allow nodes to have more than two children. Each node can have multiple branches, making multi-way trees more flexible in representing hierarchical data.

### Example:



Multi-way Tree

In this example, "A" is the root node, and it has three children, "B," "C," and "D." Node "C" has one child, "E." Node "D" has three children, "F," "G," and "H." Node "F" also has one child, "I." This is an example of a multi-way tree with different numbers of children for each node.

Understanding the different types of trees and their respective examples is essential for choosing the most suitable tree structure for specific applications in computer science and data management.

← Previous
Solution: Swap Nodes In Pairs (Easy)

Next →
Binary Search Tree (BST)

☐ Mark as Completed